MongoDB Study Notes

Hello Mahmoud. These are your detailed MongoDB study notes from the terminal

session on 2026-03-31, arranged from easy concepts to more complex ones, with a

definition first and then examples based on what you practiced today.

Prepared for Mahmoud. Focus: `mongosh`, CRUD, arrays, query operators, indexing,

and aggregation.

1. What MongoDB Is

Definition: MongoDB is a NoSQL document database. It stores data in **documents** that look similar to JSON. Groups of documents are stored inside **collections**, and collections live inside **databases**.

Main structure

- **Database:** a container for collections. Example: `new`, `blog`, `school`.

- **Collection:** a group of related documents. Example: `analytics`, `games`, `papers`.

- **Document:** one record. Example: `{ name: "mahmoud", age: 35 }`.

- **Field:** one key inside a document. Example: `name`, `age`, `address.city`.

2. Mongo Shell: Starting and Basic Use

`mongosh`

Definition: `mongosh` is the modern MongoDB shell command used to connect to

MongoDB and run database commands interactively.

Correct command:

```
mongosh
```

What happened on your machine: `mongosh` opened successfully and connected to `mongodb://127.0.0.1:27017`. Your MongoDB server version shown in the shell was `8.2.6`.

3. Databases and Collections

```
use <db>
```

Definition: `use` switches the current database. After switching, `db` points to that database.

Correct command:

```
use new
```

After that, your shell prompt became `new>`, which means you were inside the database

`new`.

`db`

Definition: `db` is the current database object in `mongosh`. You use it to access

collections and run commands.

`show dbs` and `show collections`

Definition:

- `show dbs` lists databases.

- `show collections` lists collections in the current database.

Correct command:

```
show collections
```

When you ran it correctly in database `new`, you saw collections such as `analytics`,

`blog`, `games`, `movies`, `papers`, and `user`.

4. Insert Operations

`insertOne()`

Definition: inserts one document into a collection.

Example from your terminal:

```
db.games.insertOne({ "game": "football", "user": "mahmoud" })
```

insertMany()

Definition: inserts multiple documents at once. It requires an array of documents.

Correct form you later used:

```
db.new.insertMany([
```

```
  { "name": "mahmoud", "age": 35 },
```

```
{ "name": "mar", "age": 20 }
```

```
])
```

Meaning: two documents were inserted into the collection named `new` inside the

database `new`.

`insert()` old style

Definition: `insert()` is an older method. It may still work, but modern code should

use `insertOne()` or `insertMany()`.

Example from your terminal:

```
for(var i=0;i<1000;i++){db.new.insert({"title":"ayhaga"})}
```

This worked after fixing the syntax, but MongoDB showed a deprecation warning.

Modern version:

```
for (let i = 0; i < 1000; i++) {
```

```
db.new.insertOne({ title: "ayhaga" })
```

```
}
```

Loop insert with random data

Definition: You can use JavaScript inside `mongosh` to generate documents

automatically.

Correct example based on your terminal:

```
for (let i = 0; i < 1000; i++) {
```

```
db.user.insertOne({
```

```
  username: "user" + i,
```

```
  age: Math.floor(Math.random() * 120),
```

```
  createdAt: new Date()
```

```
})
```

```
}
```

What each part means:

- `"user" + i`: creates `user0`, `user1`, `user2`, and so on.

- `Math.random() * 120`: random number from `0` up to less than `120`.

- `Math.floor()`: removes decimals.

- `new Date()`: stores the current date and time.

5. Reading Data: `find()` and `findOne()`

`find()`

Definition: returns all matching documents from a collection. The result is a cursor.

Correct form:

```
db.user.find()
```

```
db.user.find({ username: "user101" })
```

Rule: `find()` is called on a collection, not directly on `db`.

`findOne()`

Definition: returns a single matching document instead of a cursor.

```
db.analytics.findOne({ url: "www.example.com" })
```

6. Projection

Definition: projection controls which fields are returned by `find()`.

Include fields

```
db.users.find({}, { name: 1 })
```

This includes `name`, and by default also includes `_id`.

Include fields without `_id`

```
db.users.find({}, { name: 1, _id: 0 })
```

Exclude fields

```
db.users.find({}, { location: 0 })
```

> **Rule:** In projection, `1` means include and `0` means exclude. Do not mix include and exclude in the same projection except for `_id`.

7. Updating Documents

`updateOne()`

Definition: updates the first document that matches the filter.

Correct command from your session:

```
db.new.updateOne(
```

```
{ name: "mahmoud" },
```

```
{ $set: { name: "elkholy" } }
```

```
)
```

$set

Definition: sets the value of a field. It creates the field if it does not already exist.

Examples from your terminal:

```
db.analytics.updateOne(
```

```
  { url: "www.example.com" },
```

```
  { $set: { age: 20 } }
```

```
)
```

```
db.analytics.updateOne(
```

```
  { url: "www.example.com" },
```

```
{ $set: { address: { city: "cairo", country: "egypt" } } }
```

```
)
```

Update nested field with dot notation

Definition: dot notation lets you update one field inside an embedded document.

Example from your terminal:

```
db.analytics.updateOne(
  { url: "www.example.com" },
  { $set: { "address.city": "elmofia" } }
)
```

$inc

Definition: increases or decreases a numeric field.

Examples from your terminal:

```
db.analytics.updateOne(
```

```
  { url: "www.example.com" },
```

```
{ $inc: { pageviews: 1 } }
```

```
)
```

```
db.games.updateOne(
```

```
{ game: "football", user: "mahmoud" },
```

```
{ $inc: { score: 50 } }
```

```
)
```

```
db.games.updateOne(
  { game: "football", user: "mahmoud" },
  { $inc: { score: -10 } }
)
```

8. Embedded Documents and Arrays

Definition:

- **Embedded document:** an object inside another document.

- **Array:** a list of values or objects inside a field.

Examples from your terminal:

```
{
  url: "www.example.com",
  pageviews: 11,
  age: 20,
  address: { city: "elmofia", country: "egypt" }
}
```

```
{
```

```
title: "blog",
```

```
content: "......",
```

```
comment: [
```

```
{ name: "mahmoud", age: 20 },
```

```
{ name: "amr", age: 20 }
```

```
]

}
```

9. Array Update Operators

$push

Definition: adds one value to the end of an array.

Examples from your terminal:

```
db.blog.updateOne(
  { title: "blog" },
  { $push: { comment: { name: "mahmoud", age: 20 } } }
)
```

```
db.blog.updateOne(
```

```
{ title: "blog" },
```

```
{ $push: { comment: { name: "amr", age: 20 } } }
```

```
)
```

$addToSet

Definition: adds a value to an array only if it does not already exist.

Example from your terminal:

```
db.papers.updateOne(
  { title: "research paper" },
  { $addToSet: { authors: "mahmoud" } }
)
```

$each

Definition: lets MongoDB add multiple values in one operation, usually with `$push` or `$addToSet`.

Example from your terminal:

```
db.papers.updateOne(
  { title: "research paper" },
```

```
{
```

```
$addToSet: {
```

```
"authors client": {
```

```
$each: ["mahmoud", "ali", "sara"]
```

```
}
```

```
}
```

```
}

)
```

$pull

Definition: removes array elements that match a value or a condition.

Example based on your terminal state:

```
db.movies.updateOne(
```

```
{ genre: "horror" },
```

```
{
```

```
$pull: {
```

```
top10: { $in: ["saw", "saw11", "saw111"] }
```

```
}
```

```
}
```

```
)
```

This was the correct answer to remove the string values that remained inside `top10`.

$pop

Definition: removes one element from the beginning or end of an array.

- `{ $pop: { key: 1 } }` removes the last element.

- `{ $pop: { key: -1 } }` removes the first element.

`$push` with `$each`, `$sort`, and `$slice`

Definition: this combination is used to push multiple values into an array, sort the array,

and keep only part of it.

Correct form based on what you wanted to do:

```
db.movies.updateOne(
```

```
  { genre: "horror" },
```

```
  {
```

```
    $push: {
```

```
      top10: {
```

```
        $each: [
```

```
{ name: "snow1", rate: 10 },
```

```
{ name: "snow2", rate: 3 },
```

```
{ name: "snow3", rate: 5 }
```

```
],
```

```
$sort: { rate: -1 },
```

```
$slice: 10
```

```
}

}

}

)
```

Meaning of each part:

- `$each`: adds several items.

- `$sort: { rate: -1 }`: sorts highest rate first.

- `$slice: 10`: keeps only 10 elements.

10. Query Operators

Definition: query operators are special keywords beginning with $ that allow more advanced search conditions.

Operator	Definition	Example
`$gt`	Greater than	`db.users.find({ age: { $gt: 20 } })`
`$lt`	Less than	`db.users.find({ age: { $lt: 40 } })`

`$gte`	Greater than or equal	`db.users.find({ age: { $gte: 35 } })`
`$lte`	Less than or equal	`db.users.find({ age: { $lte: 35 } })`
`$ne`	Not equal	`db.users.find({ name: { $ne: "ali" } })`
`$in`	Matches any value from a list	`db.users.find({ name: { $in: ["mahmoud", "ali"] } })`
`$nin`	Matches values not in a list	`db.users.find({ name: { $nin: ["mahmoud", "ali"] } })`
`$or`	At least one condition must be true	`db.users.find({ $or: [{ name: "mahmoud" }, { age: 20 }] })`
`$and`	All conditions must be true	`db.users.find({ $and: [{ age: { $gt: 20 } }, { city: "cairo" }] })`
`$not`	Negates one condition	`db.users.find({ age: { $not: { $gt: 30 } } })`
`$all`	Array must contain all given values	`db.users.find({ hobbies: { $all: ["football", "reading"] } })`
`$size`	Array has exact length	`db.movies.find({ top10: { $size: 10 } })`

`$all`

Definition: used for arrays. It checks that all specified values exist in the array.

```
db.items.find({
  fruits: { $all: ["apple", "banana"] }
})
```

$size

Definition: matches arrays of exactly a given length.

```
db.movies.find({ top10: { $size: 10 } })
```

11. Sorting and Limiting Results

`sort()`

Definition: orders query results.

- `1` means ascending.

- `-1` means descending.

Example based on your question:

```
db.users.find({ age: 21 }).sort({ username: -1 })
```

This means: find users with age 21, then sort those results by `username` descending.

`limit()`

Definition: returns only a limited number of results.

Example from your terminal:

```
db.user.find({ username:
```

```
"user101" }).limit(1).explain("executionStats")
```

Important lesson from today: `limit(1)` reduced how many documents MongoDB

returned, but without an index MongoDB still used `COLLSCAN`. So `limit()` is not a

replacement for indexing.

12. Indexing

What an index is

Definition: an index is a special data structure that helps MongoDB find documents

faster based on one or more fields.

Main idea:

•Without index: MongoDB may scan all documents.

•With index: MongoDB can jump through indexed values and search faster.

`createIndex()`

Definition: creates an index on a field or a set of fields.

```
db.user.createIndex({ username: 1 })
```

Meaning: create an ascending index on `username`.

`ensureIndex()`

Definition: old MongoDB shell method for creating an index. It still appeared in your

session, but modern syntax should use `createIndex()`.

```
db.users.ensureIndex({ username: 1 })
```

Important mistake from your terminal: you inserted the 1000 random documents into `db.user`, but then you created the index on `db.users`. Those are different collections. To optimize the query on the data you inserted, the index must be created on `db.user`.

How to test index use: `explain("executionStats")`

Definition: `explain()` shows how MongoDB executed the query.

Example from your terminal:

```
db.user.find({ username: "user101" }).explain("executionStats")
```

What your output showed before indexing:

- `stage: 'COLLSCAN'`

- `totalKeysExamined: 0`

- `totalDocsExamined: 1000`

Meaning:

- `COLLSCAN`: full collection scan.

- `totalKeysExamined: 0`: no index was used.

- `totalDocsExamined: 1000`: MongoDB checked all 1000 documents.

What you want after adding the right index:

- `IXSCAN` instead of `COLLSCAN`
- very small `totalDocsExamined`
- non-zero `totalKeysExamined`

Advantages of indexing

- Faster queries.

- Faster filtering by indexed fields.

- Can help sorting.

- Can reduce full collection scans.

Disadvantages of indexing

- Slower insert, update, and delete operations.

- Extra disk space usage.

- Extra RAM usage.

- If there are too many indexes, write performance drops.

Accurate wording based on your question: MongoDB does not rewrite all indexes

fully on every change. It updates the relevant index entries affected by that changed document.

How to choose fields for indexes

Definition: choose fields that are frequently used in real queries, not every field.

- Index fields used often in `find()`.

- Index fields used in `sort()`.

- Index fields that strongly reduce searched documents, such as `username`, `email`, IDs.

•Do not index everything.

Compound index

Definition: an index on more than one field.

```
db.users.createIndex({ age: 1, username: -1 })
```

Good matching query from what you asked:

```
db.users.find({ age: 21 }).sort({ username: -1 })
```

Important rule: compound indexes follow the **leftmost prefix rule**. MongoDB benefits most when the query starts from the first field in the index.

Nested field index

Definition: MongoDB can index fields inside embedded documents using dot notation.

```
db.posts.createIndex({ "comment.date": 1 })
```

If `comment` is an embedded document, this indexes its `date` field.

Multikey index

Definition: when you create an index on an array field, MongoDB automatically creates

a multikey index. This means one document can create multiple index entries, one for each array

element.

Example:

```
db.users.createIndex({ hobbies: 1 })
```

If a document is:

```
{ name: "mahmoud", hobbies: ["football", "reading", "swimming"] }
```

MongoDB indexes:

- `"football"`

- `"reading"`

- `"swimming"`

Relation to your question: yes, indexing an array means index entries are created for

each element, and if the array changes, MongoDB updates the relevant multikey entries.

13. Aggregation

What aggregation is

Definition: aggregation is a framework in MongoDB used to process documents through

ordered stages and produce computed or transformed results.

Common uses:

- grouping

•counting

•sorting

•filtering

•reshaping output

`aggregate()`

Definition: runs a pipeline made of stages such as `$match`, `$group`, `$project`,

`$sort`, and `$limit`.

`$project`

Definition: chooses fields or reshapes documents during aggregation.

```
{ $project: { author: 1, _id: 0 } }
```

$group

Definition: groups documents by a key and computes summary values, such as counts or sums.

```
{
  $group: {
    _id: "$author",
```

```
count: { $sum: 1 }
```

```
}
```

```
}
```

`$sort` in aggregation

Definition: sorts the pipeline result.

```
{ $sort: { count: -1 } }
```

`$limit` in aggregation

Definition: keeps only the first N results after the previous stages.

```
{ $limit: 5 }
```

`$unwind`

Definition: breaks an array field into separate documents, one per array element. This is

needed when you want to count individual authors from an `authors` array.

Your aggregation example

You inserted:

```
db.article.insertMany([
```

```
{
  title: "article1",
  author: { id: "r.l.st" }
},
{
  title: "article2",
```

```
        author: { id: "grotdsad" }
```

```
}
```

```
])
```

Then you grouped by author with:

```
db.article.aggregate([
```

```
{ $project: { author: 1, _id: 0 } },
```

```
{
```

```
$group: {
```

```
_id: "$author",
```

```
count: { $sum: 1 }
```

```
}
```

```
}

])
```

Result meaning: MongoDB grouped the documents by the full `author` object and counted how many times each one occurred.

Top 5 authors from an authors array

This answers the task you asked about:

```
db.articles.aggregate([
```

```
  { $unwind: "$authors" },
```

```
  {
```

```
    $group: {
```

```
      _id: "$authors",
```

```
      count: { $sum: 1 }
```

```
}

},

{ $sort: { count: -1 } },

{ $limit: 5 }

])
```

Stage-by-stage meaning:

1. `$unwind`: take each author out of the array one by one.

2. `$group`: group equal author names together.

3. `$sum: 1`: count each occurrence.

4. `$sort`: sort by count descending.

5. `$limit`: keep only the top 5.

14. The Meaning of $ in Different Positions

Definition: the symbol `$` has different meanings depending on where it appears.

- **Operator name:** `$set`, `$inc`, `$gt`, `$push`.

- **Field reference in aggregation:** `"$author"` means the value of the field `author`.

- **Array positional syntax:** examples like `$[]` or `$[elem]` are used in advanced array

updates.

15. Practical Summary of What You Learned Today

- How to start `mongosh`.

- How to switch databases with `use`.

- How to insert one or many documents.

- How to read documents with `find()`.

•How projection works.

- How to update with `$set` and `$inc`.

- How arrays are updated with `$push`, `$addToSet`, `$pull`, `$pop`.

- How query operators such as `$in`, `$or`, `$all`, and `$size` work.

•How to sort and limit results.

- Why indexing matters and how to read `COLLSCAN` versus `IXSCAN`.

•What compound and multikey indexes are.

- How aggregation works using `$project`, `$group`, `$sort`, `$limit`, and `$unwind`.

---

16. Final Cheat Sheet

```
mongosh
```

```
use new
```

```
show collections
```

```
show dbs
```

```
db.user.insertOne({ username: "user1", age: 20 })
```

```
db.user.insertMany([
```

```
  { username: "user2", age: 21 },
```

```
  { username: "user3", age: 22 }
```

```
])
```

```
db.user.find()
```

```
db.user.find({ username: "user1" })
```

```
db.user.find({}, { username: 1, _id: 0 })
```

```
db.user.updateOne(
```

```
  { username: "user1" },
```

```
  { $set: { age: 21 } }
```

```
)
```

```
db.user.updateOne(
```

```
{ username: "user1" },
```

```
{ $inc: { age: 1 } }
```

```
)
```

```
db.blog.updateOne(
```

```
{ title: "blog" },
```

```
{ $push: { comment: { name: "mahmoud", age: 20 } } }
```

```
)
```

```
db.papers.updateOne(
```

```
{ title: "research paper" },
```

```
{ $addToSet: { authors: { $each: ["mahmoud", "ali"] } } }
```

```
)
```

```
db.user.createIndex({ username: 1 })
```

```
db.user.find({ username: "user101" }).explain("executionStats")
```

```
db.articles.aggregate([
```

```
  { $unwind: "$authors" },
```

```
{ $group: { _id: "$authors", count: { $sum: 1 } } },
```

```
{ $sort: { count: -1 } },
```

```
{ $limit: 5 }
```

```
])
```

End of detailed notes. This file was built from the commands, errors, corrections, and explanations discussed

in the terminal session.